2026-11-05 글, 캐시, 로그인 토큰

- 날짜: 2026-11-05
- 앱: `lgcns-fe-next`
- 패키지: `next` 16.4.3, `react` 19.3.0, `next-auth` ^4.24.15, `jose`, `quill` 2.0.3, `sanitize-html`, Prisma 6 + MongoDB
- 기준: 이 문서의 "현재 코드"는 지금 디스크의 파일이다. 시크릿, 토큰 문자열, 데이터베이스 URL은 적지 않는다.
- 비용 추정은 여기 없다. 그 비교는 `docs/2026-11-05-post-cache-scale.md`에 따로 있다.

오늘 만든 기능은 네 영역이다. 글 데이터, 그 데이터를 읽는 Next 캐시, 작성·수정·댓글·좋아요 화면, 로그인 JWT와 소셜 계정 연결이다.

1. 파일이 언제 실행되는가

이 앱 소스에는 글 기능을 불러오는 `<script src>`가 없다. Next가 서버 모듈과 `"use client"` 모듈을 나눠 번들한다.

HTML에서 `async`는 받는 즉시 실행하고 순서를 보장하지 않는다. `defer`는 파싱이 끝난 뒤 문서에 적힌 순서대로 실행한다. `type="module"`은 기본적으로 지연되는 ES 모듈이다. 아래 파일은 그 속성으로 붙지 않는다.

파일	선언	실행 시점	<script>와의 차이
lgcns-fe-next/proxy.ts	export async function proxy	매칭된 요청마다, 페이지를 그리기 전에 Node에서 실행. JWT 서명을 await verifyAccessToken으로 확인	브라우저로 내려가지 않는다. 문서 파싱과 무관하다
lgcns-fe-next/lib/postCache.ts	export async function getCachedPosts / getCachedPost. 함수 첫 줄 "use cache"	목록·상세 서버 렌더가 이 함수를 부를 때. 캐시 히트면 함수 본문의 Prisma는 다시 실행되지 않는다	서버 함수다. async 속성의 스크립트가 아니다
lgcns-fe-next/service/postService.ts	export const postService	캐시 함수가 미스로 DB를 읽을 때, 그리고 서버 액션이 쓸 때	서버 전용 모듈. 클라이언트 번들에 넣지 않는다
lgcns-fe-next/actions/postActions.ts	파일 첫 줄 "use server"	폼이 제출될 때 그 페이지로의 POST로 실행	클릭 즉시 브라우저에서 도는 함수가 아니다
lgcns-fe-next/app/(post)/posts/page.tsx	export default function PostsPage. searchParams는 Promise	프록시가 통과시킨 뒤 서버에서 렌더. searchParams를 읽는 자식은 <Suspense> 안	페이지 HTML을 서버가 만든다
lgcns-fe-next/component/organisms/PostCard.tsx	첫 줄 "use client"	서버가 카드 props를 보낸 뒤 브라우저가 hydrate. 더 보기 상태는 useState	Next 클라이언트 모듈이다. defer로 순서를 맞추는 스크립트가 아니다
lgcns-fe-next/component/organisms/PostEditor.tsx	"use client". Quill은 useEffect 안의 await import("quill")	작성 화면이 hydrate된 뒤, effect가 브라우저에서 Quill 모듈을 받은 다음에 에디터를 만든다	동적 import()는 그 시점의 네트워크 요청이다. 문서의 <script type="module"> 태그는 없다
lgcns-fe-next/component/organisms/PostLiveRefresh.tsx	"use client"	hydrate 후 useEffect가 5초 간격으로 router.refresh()	window.setInterval은 브라우저 타이머다. 서버 캐시 지시어가 아니다
lgcns-fe-next/lib/accessJwt.ts	export async function signAccessToken / verifyAccessToken	로그인 콜백이 서명할 때, 프록시·AuthShell·readCurrentUser가 검증할 때	jose의 WebCrypto 서명. 브라우저에 비밀키를 보내지 않는다

파일	선언	실행 시점	<script>와의 차이
lgcns-fe-next/lib/authOptions.ts	export const authOptions	NextAuth 라우트와 getServerSession이 import 할 때 서버에서 평가	proxy.ts는 이 파일을 import하지 않는다. 프록시는 쿠키의 JWT만 검증한다

참고: HTML script 요소, MDN script, MDN import(), Next.js use cache (로컬 사본 node_modules/next/dist/docs/01-app/03-api-reference/01-directives/use-cache.md).

2. 데이터

스키마는 lgcns-fe-next/prisma/schema.prisma다. MongoDB다.

모델	역할
User	회원. password는 소셜 전용 회원이면 비울 수 있다. accounts로 소셜 계정을 가리킨다
Account	provider + providerAccountId가 유일하고, userId로 User에 연결된다
Post	authorId, authorName, HTML body, createdAt
Comment	postId로 글을 찾고, 작성자 이름과 본문을 저장한다
Like	postId + userId가 유일하다. 한 사용자는 글마다 좋아요가 하나다

글 본문은 Prisma 모델 인스턴스 그대로 캐시에 넣지 않는다. postService가 createdAt을 ISO 문자열로 바꾼 평범한 객체를 반환한다. use cache의 반환값은 직렬화할 수 있어야 한다.

참고: Prisma MongoDB, Next.js use cache 반환값.

3. 캐시

next.config.ts는 cacheComponents: true, partialPrefetching: true다. 이 설정이 있어야 "use cache", cacheTag, cacheLife, updateTag가 동작한다.

3.1 목록과 상세를 캐시에서 읽기

lib/postCache.ts의 두 함수가 캐시 경계다.

- getCachedPosts(pageNumber, sortOrder)는 cacheTag("posts"), cacheLife("max") 뒤에 postService.listPosts를 호출한다.
- getCachedPost(postId)도 같은 태그와 같은 수명으로 postService.getPost를 호출한다.
- 인자 pageNumber, sortOrder, postId는 캐시 키에 들어간다. 1페이지 최신순과 2페이지 오래된순은 서로 다른 항목이다.
- 함수 안의 console.log는 그 항목을 채울 때 실행된다. 같은 키의 다음 호출은 저장된 결과를 돌려준다.

cacheLife("max")의 프리셋은 stale 5분, 서버 재검증 30일, 만료 1년이다. 이 프로젝트의 next.config.ts는 이 프리셋을 다시 정의하지 않는다. 쓰기가 updateTag("posts")를 호출하면 그 수명과 관계없이 태그가 바로 만료된다.

postService.listPosts는 한 페이지 50개다. createdAt으로 정렬하고 skip/take로 자른 뒤, 그 50개의 댓글과 좋아요만 이어서 읽는다. 댓글 본문도 목록 캐시 항목에 들어 있다. 화면의 더보기는 이 배열을 접고 펼친다. 더보기를 누를 때마다 데이터베이스를 다시 읽지 않는다.

목록 페이지는 searchParams를 캐시 함수 안에서 읽지 않는다. posts/page.tsx의 PostsContent가 page와 sort를 숫자와 "ASC" | "DESC"로 바꾼 뒤 인자로 넘긴다. searchParams는 요청마다 다른 런타임 값이라 "use cache" 안에서 직접 읽으면 안 된다. 그래서 PostsContent는 <Suspense> 안에 있다.

PostListSection과 Pagination은 같은 postList 프로미스에 await를 건다. 페이지는 getCachedPosts를 한 번만 호출한다.

사용자 조회는 캐시 밖에 있다. PostListSection은 Promise.all로 readCurrentUser()와 글 목록 프로미스를 같이 기다린다. 글 목록이 캐시 히트여도 사용자 한 건은 데이터베이스를 본다. 이 조회는 수정·삭제 버튼을 작성자에게만 보여 주려고 있다.

3.2 쓰면 태그를 비운다

actions/postActions.ts의 작성, 수정, 삭제, 좋아요, 댓글은 저장이 성공한 뒤에만 updateTag("posts")를 호출한다. 실패해서 return하거나 다른 경로로 redirect하기 전에는 호출하지 않는다.

updateTag는 서버 액션 안에서만 호출할 수 있다. 이 태그를 즉시 만료시켜서, 다음 읽기는 갱신된 값을 기다린다. 목록과 상세가 태그 "posts" 하나라서, 좋아요 한 번이 목록과 모든 글 상세 캐시를 함께 지운다.

PostLiveRefresh는 보이는 탭에서 5초마다 router.refresh()를 호출한다. 이것은 서버 컴포넌트를 다시 그리라는 요청이다. 태그가 살아 있으면 getCachedPosts는 캐시를 반환한다. 그 사이에 다른 사람이 글을 썼다면 태그가 이미 비워져 있어 다음 refresh가 데이터베이스를 다시 읽는다.

참고: Caching, cacheLife, updateTag, cacheComponents. 로컬 사본은 node_modules/next/dist/docs/01-app/ 아래 같은 경로다.

4. 글 화면

라우트 그룹 (post)는 URL에 들어가지 않는다. app/(post)/layout.tsx가 Header를 붙인다.

주소	파일	하는 일
/posts	app/(post)/posts/page.tsx	최신순·오래된순, 페이지, 글 목록. 글쓰기 링크는 /postwrite
/postdetail/[postId]	app/(post)/postdetail/[postId]/page.tsx	본문 전체, 댓글, 좋아요. 없으면 notFound()
/postwrite	app/(post)/(postsys)/postwrite/page.tsx	쿼리에 postId가 없으면 작성, 있으면 수정
/postwrite/[postId]	app/(post)/(postsys)/postwrite/[postId]/page.tsx	/postwrite?postId=로 redirect만 한다

4.1 목록에서 개수만 보이고, 더보기로 댓글이 열린다

PostCard는 "use client"다. 목록에서는 excerpt로 태그를 벗긴 앞 140자를 보여 준다. 좋아요 수와 댓글 수는 캐시에 들어 있는 숫자다. useState(false)인 더보기를 누르면 같은 props의 comments를 렌더한다.

상세는 full이라 본문이 <로 시작하면 dangerouslySetInnerHTML로 Quill HTML을 그린다. 목록의 요약은 그 속성을 쓰지 않는다. 저장 전에 sanitizePostHtml이 허용한 태그만 남긴다.

4.2 좋아요, 댓글, 삭제

카드의 `<form action={toggleLike}>`, `createComment`, `deletePost`는 서버 액션이다. 폼에는 `postId`만 숨은 필드로 넣고, 사용자 아이디 필드는 없다. 삭제는 제출 전에 `window.confirm`으로 브라우저 확인을 받는다. 확인을 취소하면 `preventDefault`로 액션을 호출하지 않는다.

`postService.toggleLike`는 `postId_userId` 유니크로 기존 행을 찾고, 있으면 지우고 없으면 만든다. `createComment`는 본문 trim 후 1자 이상 300자 이하여야 하고, 글이 있을 때만 저장한다. `deletePost`는 작성자가 아니면 `forbidden`이고, 맞으면 댓글, 좋아요, 글 순서로 지운다.

5. 작성과 수정은 한 페이지

`/postwrite`와 `/postwrite?postId=`가 같은 `page.tsx`다. `searchParams`는 `Promise`라서 `PostWriteContent`가 `<Suspense>` 안에서 `await`한다.

- `postId`가 없으면 제목은 "글 작성", 액션은 `createPost`, 버튼은 "올리기"다.
- `postId`가 있으면 `getCachedPost`로 본문을 채운다. 글이 없으면 `/posts`로 보낸다. 작성자가 아니면 `/postdetail/[postId]`로 보낸다. 액션은 `updatePost`, 버튼은 "수정하기"다.
- 이 페이지도 `readCurrentUser()`를 호출한다. 로그인 쿠키가 없거나 토큰이 검증되지 않으면 `/login?callbackUrl=`로 보낸다. 프록시가 `/postwrite`를 이미 막고, 제출 시 액션이 다시 토큰을 확인한다. 페이지의 조회는 에디터를 보여 주기 전에 작성자인지를 나누려고 있다.

5.1 Quill

`PostEditor`의 `QuillBox`는 `memo`다. 글자를 입력할 때마다 부모 state가 바뀌어도 에디터 DOM을 React가 다시 비우지 않게 하려고 props를 `initialHtml`과 `onChange`로 한정한다. `onChange`는 ref에 넣어 effect가 다시 돌지 않게 한다.

Quill 생성은 `useEffect` 안에서 `await import("quill")` 다음이다. 서버 렌더에서는 `document`가 없어 생성자를 호출하지 않는다. 툴바는 제목, 굵게, 기울임, 밑줄, 취소선, 순서 있는 목록, 순서 없는 목록, 인용, 링크, 서식 지우기다. 변경분은 `getSemanticHTML()`로 숨은 input `body`에 넣는다. 보이는 글자가 없으면 제출 버튼이 비활성이다.

`quill/dist/quill.snow.css`는 클라이언트 모듈의 정적 import다. `next.config.ts`의 Turbopack CSS 규칙은 `condition: { not: "foreign" }`라서 `node_modules`의 이 CSS는 Tailwind 로더를 타지 않는다.

5.2 저장되는 HTML

`lib/postHtml.ts`의 `sanitizePostHtml`은 `p`, `br`, `strong`, `em`, `u`, `s`, `a`, `ol`, `ul`, `li`, `h1~h3`, `blockquote`, `span`만 남긴다. 링크 스킴은 `http`, `https`, `mailto`다. 목록 들여쓰기 클래스 `ql-indent-`숫자만 허용한다. `postText`는 태그를 모두 뺀 글자다. 비어 있거나 5000자를 넘으면 `createPost`/`updatePost`는 저장하지 않고 `updateTag`도 호출하지 않는다.

참고: React useEffect, React memo, MDN FormData, Quill. `dangerouslySetInnerHTML`은 저장 전에 걸러 둔 HTML을 상세에서만 쓴다. React DOM.

6. 로그인 토큰

글 폼은 사용자 아이디를 보내지 않는다. 서버가 로그인 때 발급한 JWT를 검증하고, 그 토큰의 `sub`로 회원을 찾는다.

6.1 발급

`lib/accessJwt.ts`는 `jose`의 `SignJWT`로 HS256 토큰을 만든다. 비밀은 `NEXTAUTH_SECRET`이다. `sub`는 MongoDB 사용자 id이고, 클레임에 email과 name이 있다. 만료는 2시간(`7200`초)이고 NextAuth `session.maxAge`와 같다.

- 아이디·비밀번호 로그인: `userService.login`이 비밀번호를 확인한 뒤 `signAccessToken`을 호출한다. 비밀번호가 없는 소셜 전용 회원은 이 경로로 들어오지 못한다. `authOptions`의 credentials `jwt` 콜백이 그 문자열을 `token.accessToken`에 넣는다.
- 카카오·구글: `jwt` 콜백이 `account`가 있을 때만 `userService.linkSocialAccount`를 호출하고, 연결 결과의 id로 같은 `signAccessToken`을 만든다. 카카오 인가 URL은 `profile_nickname,profile_image,account_email` 스코프를 요청한다. 공급자 access token은 우리 세션에 넣지 않는다.

`POST /api/auth/login`이 돌려주는 `accessToken`도 이 JWT다. 예전처럼 `randomBytes`로 만든 임의 문자열은 서명이 없어 `verifyAccessToken`이 거절한다.

6.2 검증

`verifyAccessToken`은 서명, 만료, `sub`가 24자리 16진인지를 본다. 실패하면 `null`이다.

호출하는 곳	쓰는 결과
`proxy.ts`	서명이 있는 토큰만 로그인으로 본다. `/postwrite`와 그 하위는 `authPrefixList`라서 토큰이 없으면 `/login?callbackUrl=`로 보낸다
`AuthShell`	`getServerSession`의 `accessToken`을 검증해서, 통과한 문자열만 `AuthProvider`에 넘긴다
`AuthProvider`	`"use client"`. `accessToken`이 있으면 `isLoggedIn`이다. `LoginStatus`가 이 값으로 로그아웃 버튼을 그린다
`readCurrentUser`	쿠키의 NextAuth JWT에서 `accessToken`을 꺼내 검증한 뒤 `userService.findSessionUser`로 이름과 이메일을 읽는다

`readCurrentUser`는 `await connection()` 다음에 `cookies()`를 읽는다. Cache Components에서 쿠키는 요청 시점 값이라, 이 호출은 `"use cache"` 밖에 둔다.

6.3 소셜 계정을 기존 회원에 연결

`linkSocialAccount` 순서는 이렇다.

1. `provider` + `providerAccountId`로 `Account`를 찾으면 그 `User`를 쓴다. 이메일이 `@oauth.local`로 끝나고, 새로 받은 이메일을 다른 회원이 쓰지 않으면 그 이메일로 바꾼다.
2. 연결이 없고 이메일이 있으면 그 이메일의 `User`를 찾아 `Account` 행만 추가한다.
3. 둘 다 없으면 `User`와 `Account`를 같이 만든다. `userId`는 구글 `g_`, 그 외 `k_`에 sha256 앞 12글자를 붙인다. 이메일이 없으면 `그아이디@oauth.local`이다. `password`는 넣지 않는다.

이미 연결된 계정의 이메일이 다른 회원의 이메일과 같아도 계정을 옮기지 않는다. 이름과 프로필 이미지는 비어 있을 때만 채운다.

참고: RFC 7519 JWT, jose SignJWT, NextAuth JWT callback, MDN SubtleCrypto. NextAuth 세션 전략 `jwt`는 서버 세션 테이블 없이 쿠키에 토큰을 둔다. NextAuth session strategy.

7. 글 작업이 사용자를 확인하는 곳

서버 액션의 `author()`가 `readCurrentUser()`를 호출한다. 사용자가 없으면 각 액션의 `callbackPath`로 로그인에 보낸다. 작성 경로는 `/postwrite`, 수정 경로는 `/postwrite?postId=`, 삭제·좋아요·댓글은 상세 주소다.

그 다음 `postService`가 권한을 본다.

함수	거부 조건
`createPost`	본문이 비었거나 5000자를 넘음. `invalid`면 액션은 리다이렉트하지 않고 끝난다
`updatePost`	id 형식이나 본문이 잘못되면 `invalid`. 글이 없거나 `authorId`가 토큰 사용자와 다르면 `forbidden`이고 액션은 `/posts`로 보낸다
`deletePost`	같음. 성공하면 댓글과 좋아요를 먼저 지운다
`toggleLike`	id가 잘못되면 아무것도 안 한다
`createComment`	글이 없거나 본문이 1~300자가 아니면 저장하지 않는다

성공한 쓰기만 `updateTag("posts")` 후 상세나 목록으로 `redirect`한다. `redirect`는 예외를 던져서 함수를 끝내므로, 태그를 비우는 코드는 `redirect`보다 앞에 있다.

화면에서 사용자를 읽는 곳은 액션과 별개다.

- 목록 `PostListSection`은 수정·삭제 링크를 작성자에게만 그리려고 읽는다.
- 상세 `PostDetail`은 `canManage={post.authorId === user?.id}`를 계산하려고 읽는다.
- 작성 페이지는 에디터를 열기 전에 로그인과 작성자 여부를 나누려고 읽는다.

이 세 곳은 글을 저장하지 않는다. 저장 권한은 액션과 `postService`에 있다.

8. 요청이 지나가는 순서

글을 처음 열 때:

1. `proxy`가 쿠키의 NextAuth JWT에서 `accessToken`을 꺼내 서명한다. `/posts`와 `/postdetail`은 공개라 토큰이 없어도 통과한다. `/postwrite`는 토큰이 없으면 로그인으로 보낸다.
2. 루트 레이아웃의 `AuthShell`이 같은 토큰을 다시 검증하고 `AuthProvider`에 넘긴다.
3. 목록은 `searchParams`로 페이지와 정렬을 정한 뒤 `getCachedPosts`를 호출한다. 그 키의 캐시가 있으면 Prisma는 실행되지 않는다.
4. `PostListSection`이 캐시 밖에서 현재 사용자를 읽고, 카드에 `canManage`를 넘긴다.

글을 저장할 때:

1. 브라우저가 Quill HTML을 숨은 필드에 넣어 폼을 POST한다.
2. `createPost` 또는 `updatePost`가 토큰으로 사용자를 찾고, `postService`가 HTML을 거른 뒤 저장한다.
3. 성공하면 `updateTag("posts")`가 목록·상세 캐시를 만료시키고 상세로 이동한다.
4. 다음 조회가 `getCachedPosts` 또는 `getCachedPost`를 다시 채워 메모리에 둔다. 그 뒤의 같은 키 조회는 데이터베이스에 가지 않는다.